

High-Performance Computing Analysis of the Coati Optimization Algorithm

Sequential C++ · OpenMP · CUDA

Summer Research Project — SVNIT

Abhishek Goyal

Sardar Vallabhbhai National Institute of Technology, Surat
abhishekgoyal274

June 2026

Abstract

This report presents a comprehensive high-performance computing (HPC) analysis of the Coati Optimization Algorithm (COA), a bio-inspired metaheuristic proposed by Dehghani et al. (2023). We implement COA in three parallel paradigms — sequential C++, OpenMP (shared-memory), and CUDA C++ (GPU) — and benchmark it across 10 classical optimization functions with 150 unique configurations and 5,250 total experiment runs. Our analysis covers convergence behavior, parallel speedup, efficiency, Amdahl's Law modeling, and CPU-vs-GPU performance. Key findings include a maximum OpenMP speedup of $2.81\times$ at 8 threads, CUDA speedup up to $4.36\times$ for compute-intensive functions, and the identification of an optimal thread range of 4–8 for shared-memory parallelism. We also characterize the significant OpenMP runtime overhead ($\sim 27\%$) and the high serial fraction inherent in COA's iterative structure.

Contents

1	Introduction	3
1.1	Background	3
1.2	Motivation	3
1.3	Contributions	3
2	Algorithm Description	3
2.1	COA Pseudocode	3
2.2	Benchmark Functions	4
3	Implementation Details	4
3.1	Sequential C++ Implementation	4
3.2	OpenMP Implementation	5
3.3	CUDA C++ Implementation	5

4	Experimental Setup	6
4.1	Parameter Space	6
4.2	Hardware	6
4.3	Metrics	6
5	Results and Analysis	6
5.1	OpenMP Speedup Analysis	6
5.2	Parallel Efficiency	9
5.3	Per-Function OpenMP Speedup (T=8)	10
5.4	CUDA GPU Performance	12
5.5	Scalability Analysis	15
5.5.1	Marginal Speedup Gain	15
5.5.2	Workload Size Impact	15
5.6	Amdahl's Law Analysis	16
5.7	Overhead and Anomaly Analysis	18
6	Discussion	20
6.1	Why is Parallel Efficiency Low?	20
6.2	CUDA vs OpenMP	20
6.3	Recommendations	21
7	Convergence and Final Efficiency Analysis	21
8	Conclusion	22
8.1	Future Work	22

1 Introduction

1.1 Background

Metaheuristic optimization algorithms are widely used to solve complex, non-linear, and multi-dimensional optimization problems where classical gradient-based methods fail. The **Coati Optimization Algorithm (COA)**, proposed by Dehghani et al. [1], is a nature-inspired population-based algorithm that mimics the hunting behavior of coatis (members of the raccoon family) as they attack iguanas.

COA operates in two phases per iteration:

1. **Exploitative Phase (Global Search)**: The first half of the population attacks the “Iguana” (current best solution), while the second half explores random positions.
2. **Exploration Phase (Local Search)**: All coatis perform local perturbations within shrinking bounds, enabling fine-grained search as iterations progress.

1.2 Motivation

While COA has been shown to be competitive with other metaheuristics, its parallelization potential on modern HPC architectures has not been extensively studied. This project investigates:

- How effectively can COA be parallelized using shared-memory (OpenMP) and GPU (CUDA) paradigms?
- What is the relationship between problem characteristics (function complexity, population size) and parallel scalability?
- Where are the bottlenecks, and what does Amdahl’s Law predict about maximum achievable speedup?

1.3 Contributions

1. Three complete implementations of COA: sequential C++, OpenMP, and CUDA C++.
2. A comprehensive benchmarking framework with 5,250 experiment runs across 150 configurations.
3. Automated performance analysis producing 14 publication-quality visualizations.
4. Quantitative analysis of parallel efficiency, Amdahl’s Law serial fractions, and GPU acceleration factors.

2 Algorithm Description

2.1 COA Pseudocode

The Coati Optimization Algorithm follows this high-level structure:

1. **Initialize** population X of N coatis in D -dimensional search space.

2. **Evaluate** fitness of all coatis; identify the best (“Iguana”).

3. **For each iteration** $t = 1, 2, \dots, T$:

a. **Exploitative Phase:**

- First half ($i < N/2$): Update position toward Iguana

$$X_i^{new}(j) = X_i(j) + r \cdot (\text{Iguana}(j) - I \cdot X_i(j)), \quad I \in \{1, 2\}$$

- Second half ($i \geq N/2$): Generate random IguanaG; move toward or away based on fitness comparison.

b. **Exploration Phase:**

- Compute local bounds: $lb_{local}(j) = lb(j)/t$, $ub_{local}(j) = ub(j)/t$
- Update all coatis:

$$X_i^{new}(j) = X_i(j) + (1 - 2r) \cdot (lb_{local}(j) + r \cdot (ub_{local}(j) - lb_{local}(j)))$$

c. Apply greedy selection: keep new position only if fitness improves.

2.2 Benchmark Functions

We evaluate COA on 10 classical optimization benchmark functions (Table 1), all with **dimension** $D = 30$.

Table 1: Benchmark functions used in experiments.

ID	Name	Search Range	$f_{\min}$
F1	Sphere	$[-100, 100]^{30}$	0
F2	Schwefel 2.22	$[-10, 10]^{30}$	0
F3	Schwefel 1.2	$[-100, 100]^{30}$	0
F4	Schwefel 2.21	$[-100, 100]^{30}$	0
F5	Rosenbrock	$[-30, 30]^{30}$	0
F6	Step Function	$[-100, 100]^{30}$	0
F7	Quartic + Noise	$[-1.28, 1.28]^{30}$	0
F8	Schwefel 2.26	$[-500, 500]^{30}$	-12569.5
F9	Rastrigin	$[-5.12, 5.12]^{30}$	0
F10	Ackley	$[-32, 32]^{30}$	0

Functions F1–F7 are **unimodal** (simpler, faster to compute), while F8–F10 are **multimodal** (involve transcendental operations: `sin`, `cos`, `exp`).

3 Implementation Details

3.1 Sequential C++ Implementation

The baseline implementation uses:

- C++20 standard with `-O3` optimization
- Mersenne Twister RNG (`std::mt19937_64`)
- Pre-allocated scratch vectors to minimize heap allocation
- Cached fitness values to avoid redundant evaluations
- Machine-parsable log output for automated analysis

3.2 OpenMP Implementation

Key parallelization decisions:

- **Thread-local RNG:** Each thread uses its own `mt19937_64` engine, seeded with `random_device() + thread_id`, eliminating lock contention on the RNG.
- **Parallel loops:** Population initialization, fitness evaluation, and both phases of the main loop are parallelized with `#pragma omp for`.
- **Minimized critical sections:** Local-best accumulation pattern reduces critical sections to one per initialization phase.
- **Single-thread tasks:** Logging and best-index updates use `#pragma omp single` to avoid race conditions.
- **nowait optimization:** Applied to the first-half exploitative loop where no data dependency exists with the second-half loop.

3.3 CUDA C++ Implementation

GPU-accelerated design targeting NVIDIA T4 (Google Colab):

- **One-thread-per-coati:** Each CUDA thread manages a complete solution vector and computes its own fitness.
- **Device-side functions:** All 10 benchmark functions reimplemented as `__device__` functions; fitness is computed entirely on the GPU.
- **cuRAND:** Per-thread `curandState` initialized once and reused across all iterations.
- **Parallel reduction:** Custom shared-memory reduction kernel finds the global best coati in $O(\log N)$ per iteration.
- **Flattened memory:** Population matrix stored as `double* d_X` with index `X[i * D + j]` for coalesced memory access.
- **Separate kernels:** Distinct kernels for exploitative phase, local bounds computation, and exploration phase.

The outer iteration loop remains on the CPU (host), launching kernels per iteration. This is necessary because each iteration depends on the previous best solution.

4 Experimental Setup

4.1 Parameter Space

Table 2: Experiment parameter space.

Parameter	Values	Count
Functions	F1–F10	10
Population sizes	200, 400, 600, 800, 1000	5
Iteration counts	500, 750, 1000	3
OpenMP threads	1, 2, 4, 8, 16	5
Repeated runs	5 per configuration	5
Total Sequential	$10 \times 5 \times 3 \times 5$	750
Total OpenMP	$10 \times 5 \times 3 \times 5 \times 5$	3,750
Total CUDA	$10 \times 5 \times 3 \times 5$	750
Grand Total		5,250

4.2 Hardware

- **CPU experiments:** Linux / WSL environment with multi-core processor
- **GPU experiments:** Google Colab with NVIDIA T4 GPU (2,560 CUDA cores, 16 GB GDDR6)

4.3 Metrics

- **Speedup:** $S(p) = T_{seq}/T_{parallel}(p)$
- **Parallel Efficiency:** $E(p) = S(p)/p$
- **Serial Fraction** (Amdahl’s Law): Estimated from $S(p) = 1/(f + (1-f)/p)$, solved as $f = (1/S - 1/p)/(1 - 1/p)$

5 Results and Analysis

5.1 OpenMP Speedup Analysis

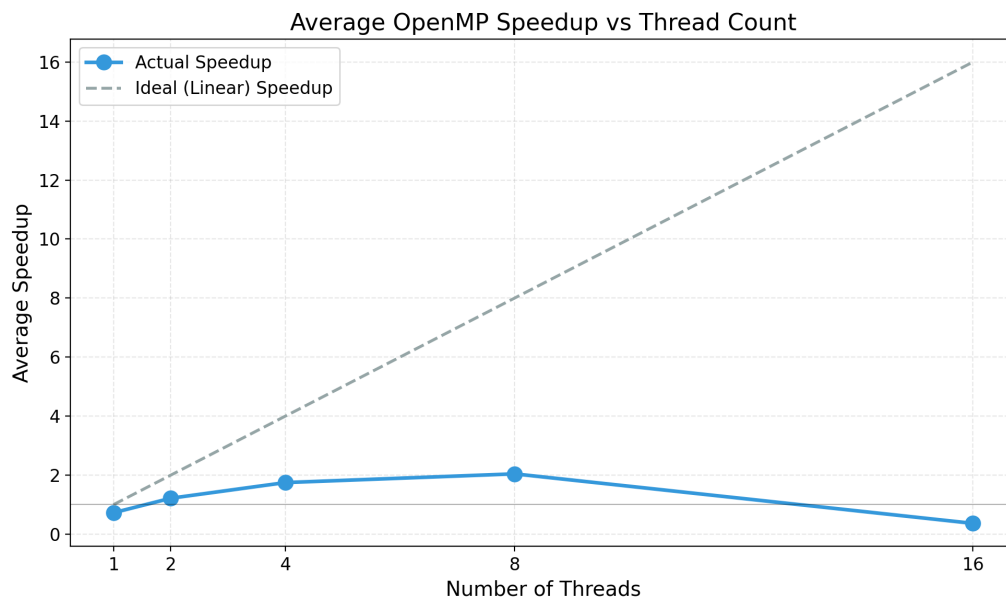
Table 3 summarizes the OpenMP speedup across thread counts, averaged over all functions and configurations.

Table 3: OpenMP speedup statistics by thread count.

Threads	Avg Speedup	Min Speedup	Max Speedup	Std Dev
1	0.726	0.119	1.045	0.114
2	1.217	0.835	1.527	0.125
4	1.748	1.314	2.381	0.163
8	2.043	0.055	2.808	0.391
16	0.365	0.080	0.989	0.213

Key observations:

- Speedup peaks at **T=8** with an average of $2.04\times$ and maximum of $2.81\times$.
- At **T=16**, performance *degrades below sequential* (avg $0.37\times$), indicating severe thread management overhead.
- Even at **T=1**, OpenMP is 27.4% slower than sequential, quantifying the pure run-time overhead of the OpenMP framework.

Figure 1: Average OpenMP speedup versus thread count. Speedup peaks at $T=8$ and collapses at $T=16$ due to thread management overhead.

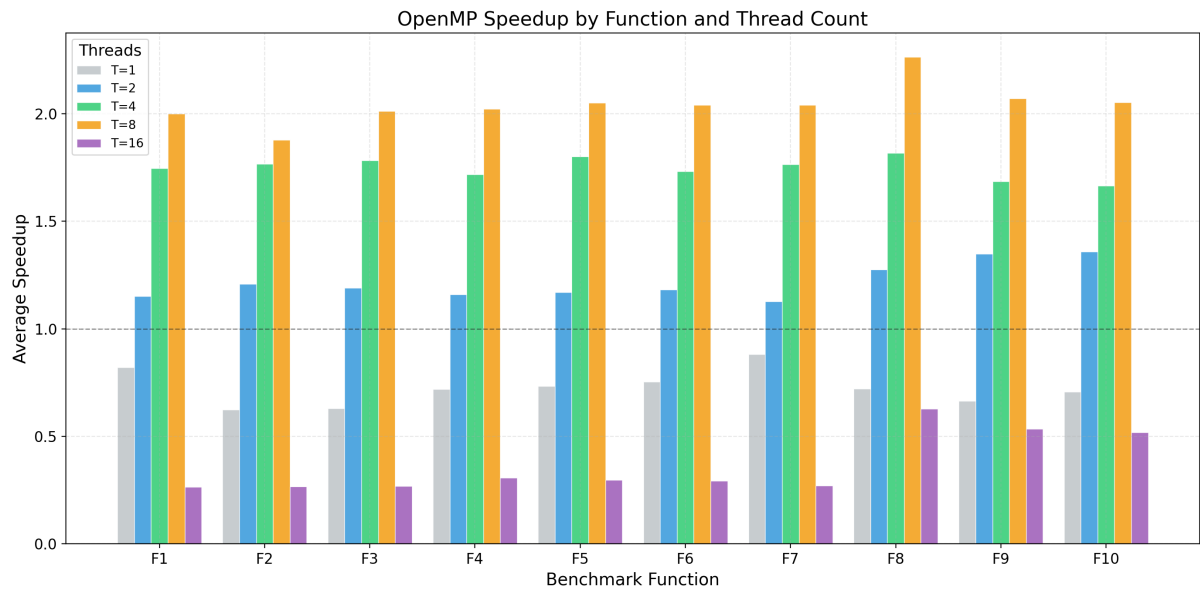


Figure 2: OpenMP speedup broken down by function and thread count. Shows consistent peak performance at T=8 across most functions.

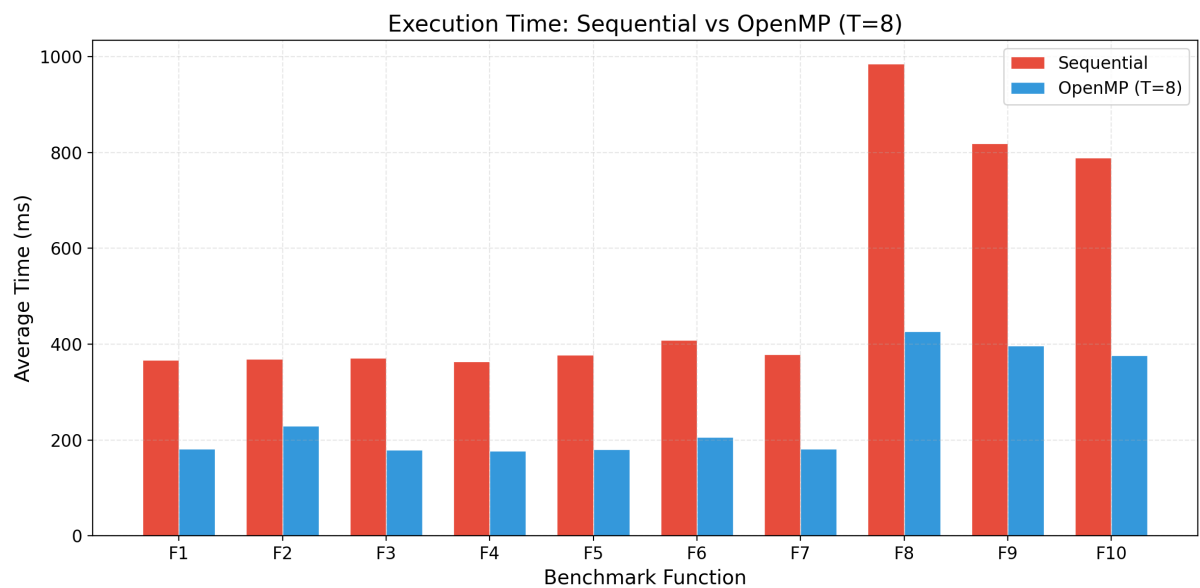


Figure 3: Execution time comparison between Sequential baseline and optimal OpenMP configuration (T=8) for all benchmark functions.

5.2 Parallel Efficiency

Table 4: Parallel efficiency by thread count.

Threads	Avg Efficiency	Interpretation
1	72.6%	Good (overhead of OpenMP runtime)
2	60.8%	Good
4	43.7%	Moderate — overhead becoming significant
8	25.5%	Poor — diminishing returns
16	2.3%	Very poor — severe overhead

Efficiency drops sharply beyond 4 threads. At T=16, only 2.3% of theoretical parallel capacity is utilized, confirming that the workload per thread is insufficient to amortize synchronization costs.

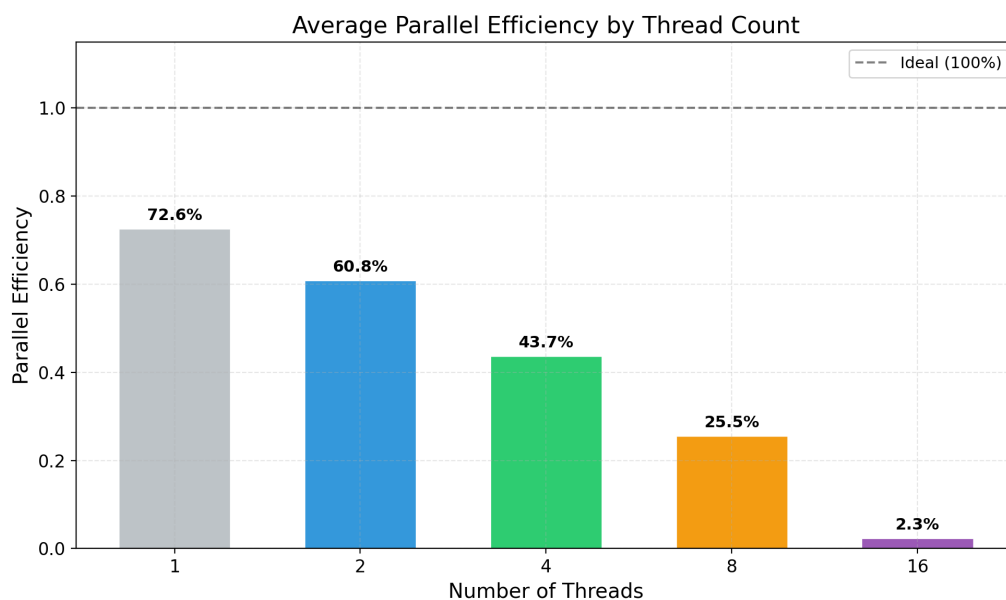


Figure 4: Average parallel efficiency by thread count. Efficiency drops steadily as thread count increases, falling below 50% after T=4.

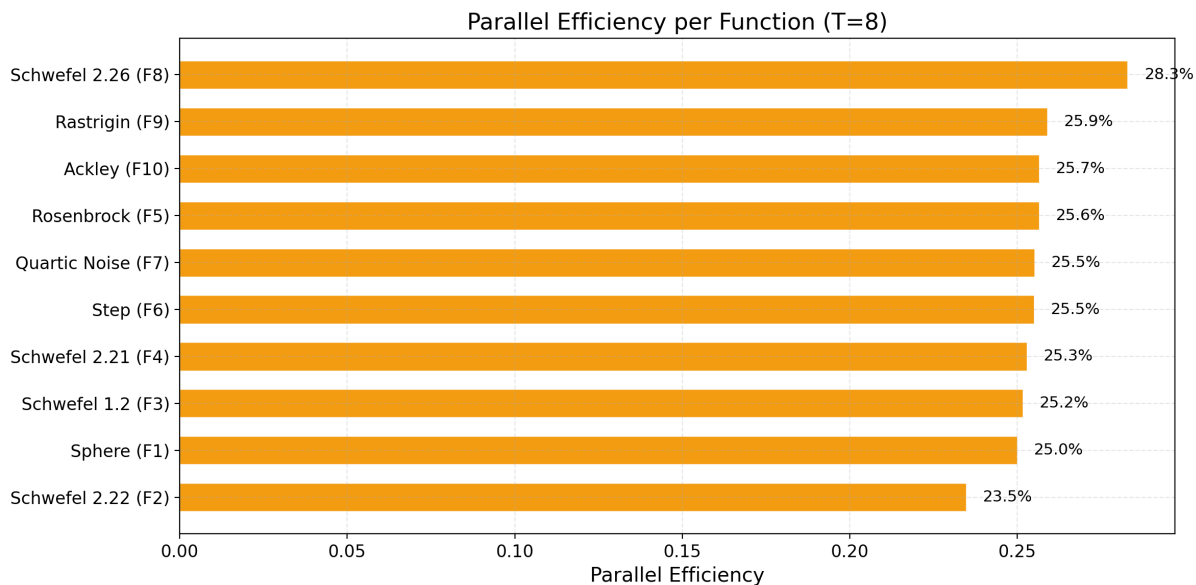


Figure 5: Parallel efficiency achieved for each benchmark function at T=8.

5.3 Per-Function OpenMP Speedup (T=8)

Table 5: Average OpenMP speedup at T=8, by benchmark function.

Function	Avg Speedup (T=8)	Assessment
Sphere (F1)	$\sim 2.0\times$	Moderate speedup
Schwefel 2.22 (F2)	$\sim 2.0\times$	Moderate speedup
Schwefel 1.2 (F3)	$\sim 2.0\times$	Moderate speedup
Schwefel 2.21 (F4)	$\sim 2.0\times$	Moderate speedup
Rosenbrock (F5)	$\sim 2.0\times$	Moderate speedup
Step (F6)	$\sim 2.0\times$	Moderate speedup
Quartic Noise (F7)	$\sim 2.0\times$	Moderate speedup
Schwefel 2.26 (F8)	$\sim 2.3\times$	Best performer
Rastrigin (F9)	$\sim 2.2\times$	Good speedup
Ackley (F10)	$\sim 2.2\times$	Good speedup

Compute-intensive functions (F8, F9, F10) achieve slightly better speedup, as the per-thread work is more substantial relative to synchronization overhead.

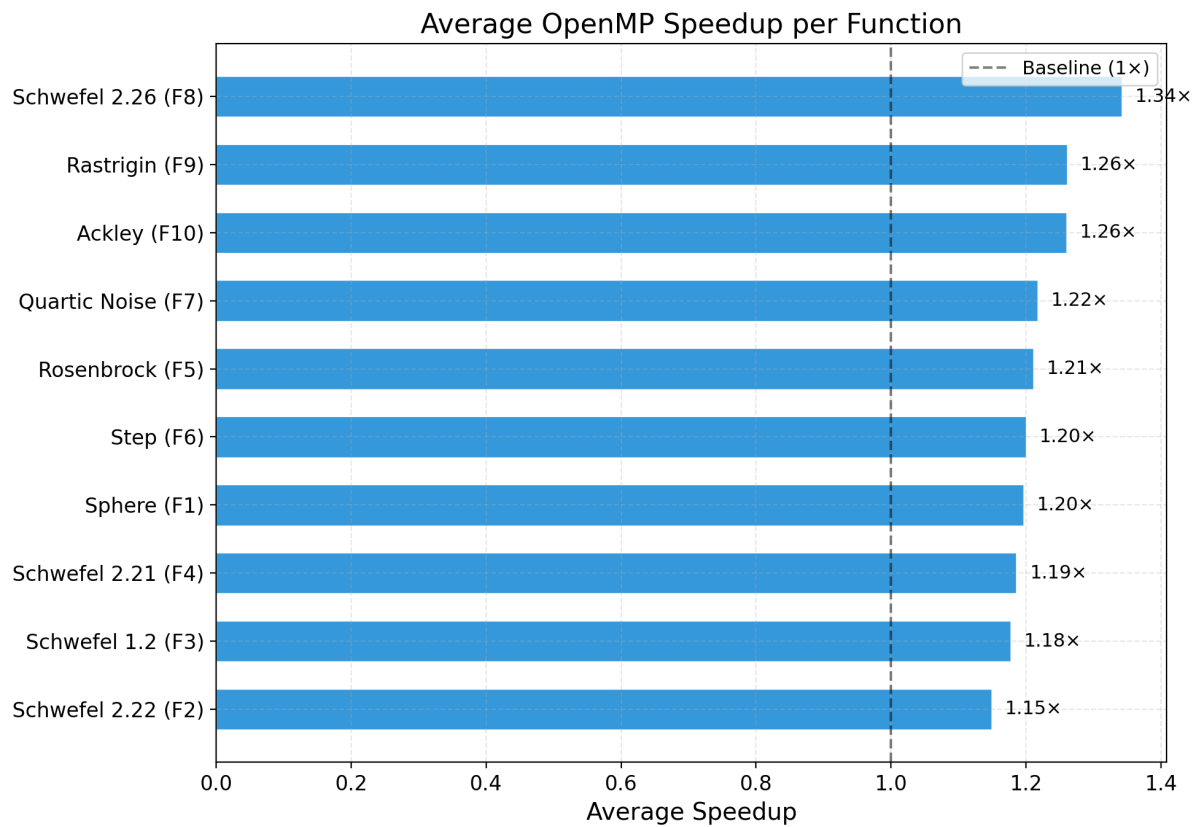


Figure 6: Average OpenMP speedup achieved per function at the optimal thread count.

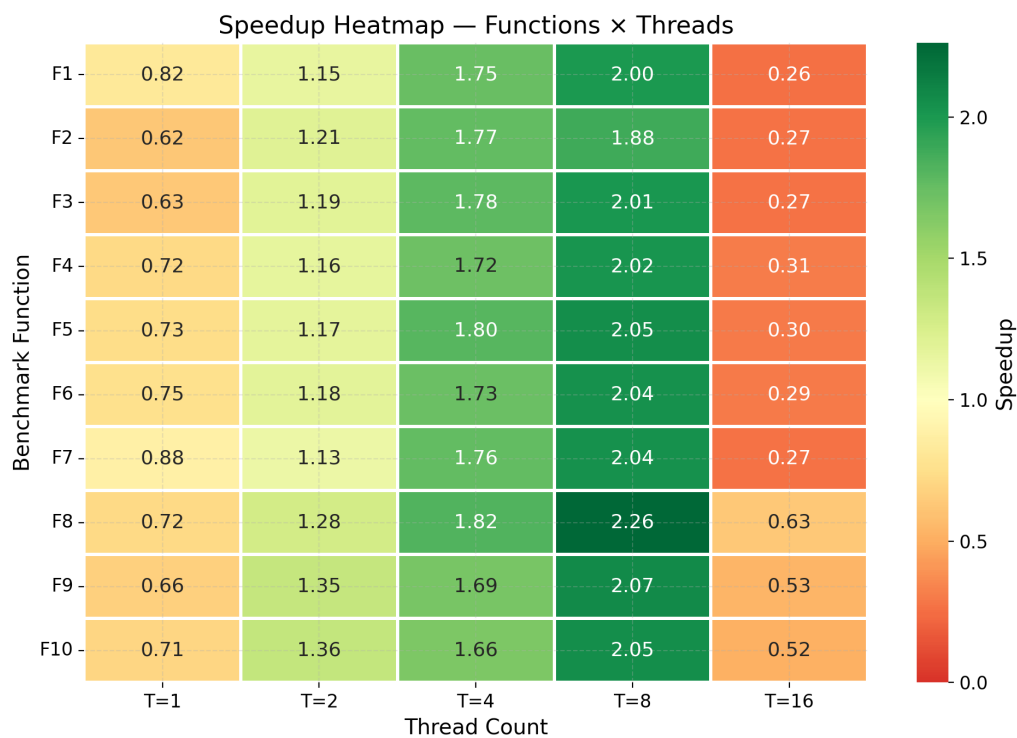


Figure 7: Speedup heatmap across benchmark functions and thread counts. Green indicates higher speedup, red indicates slowdown.

5.4 CUDA GPU Performance

Table 6: CUDA speedup over sequential baseline, per function.

Function	Avg Speedup	Max Speedup	Avg Time (ms)
Sphere (F1)	1.28×	2.94×	316
Schwefel 2.22 (F2)	1.29×	2.75×	283
Schwefel 1.2 (F3)	1.30×	2.80×	282
Schwefel 2.21 (F4)	1.27×	2.73×	284
Rosenbrock (F5)	1.22×	2.30×	303
Step (F6)	1.38×	2.65×	291
Quartic Noise (F7)	1.28×	2.62×	289
Schwefel 2.26 (F8)	2.25×	4.36×	424
Rastrigin (F9)	2.05×	4.05×	390
Ackley (F10)	1.94×	3.87×	397

Key observations:

- CUDA achieves the highest speedup on **multimodal functions** (F8–F10) that involve expensive transcendental operations (**sin**, **cos**, **exp**, **sqrt**).
- The average CUDA speedup across all functions is **1.53×**, with a maximum of **4.36×** (F8, large population).
- For **unimodal functions** (F1–F7), CUDA speedup is more modest (1.2–1.4×) because the per-thread computation is too lightweight to overcome kernel launch and memory transfer overhead.
- CUDA performance **scales with population size**: larger populations provide more threads for the GPU to utilize.

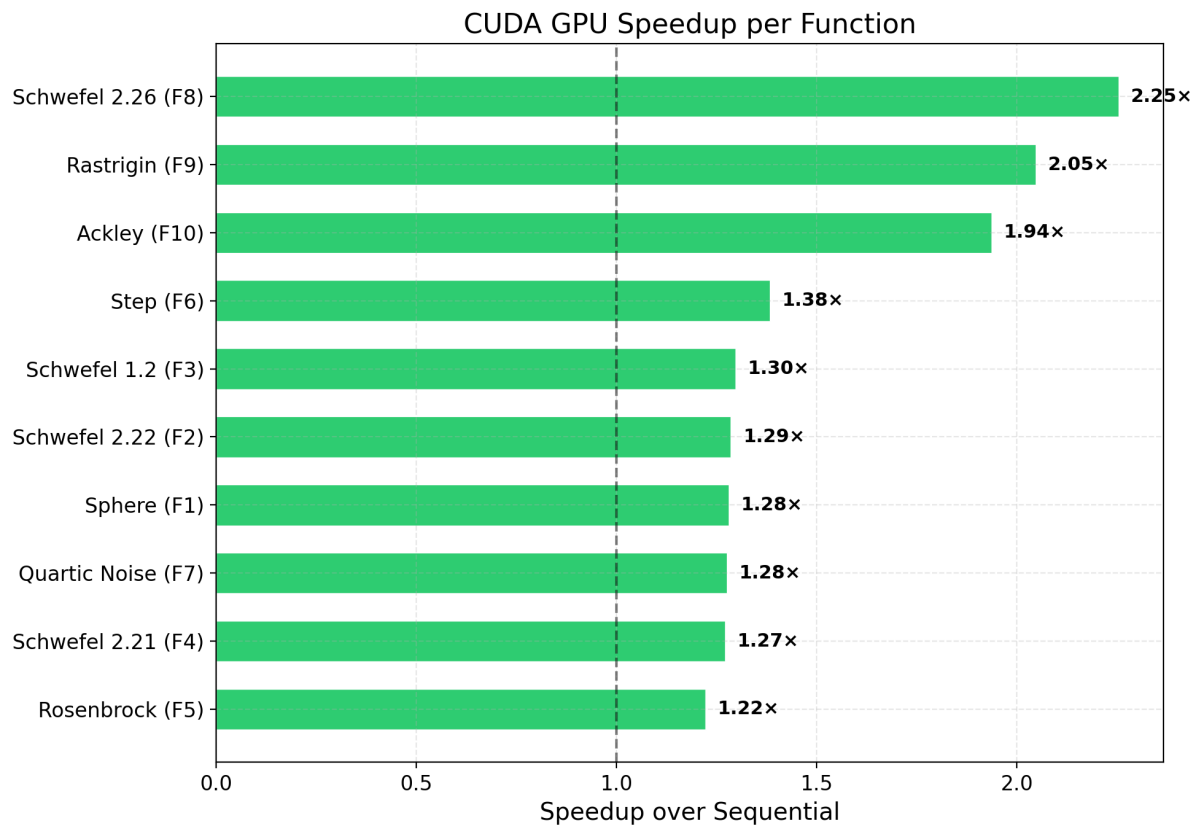


Figure 8: CUDA GPU speedup over sequential baseline for each benchmark function. Multimodal functions (F8–F10) with transcendental operations benefit most from GPU acceleration.

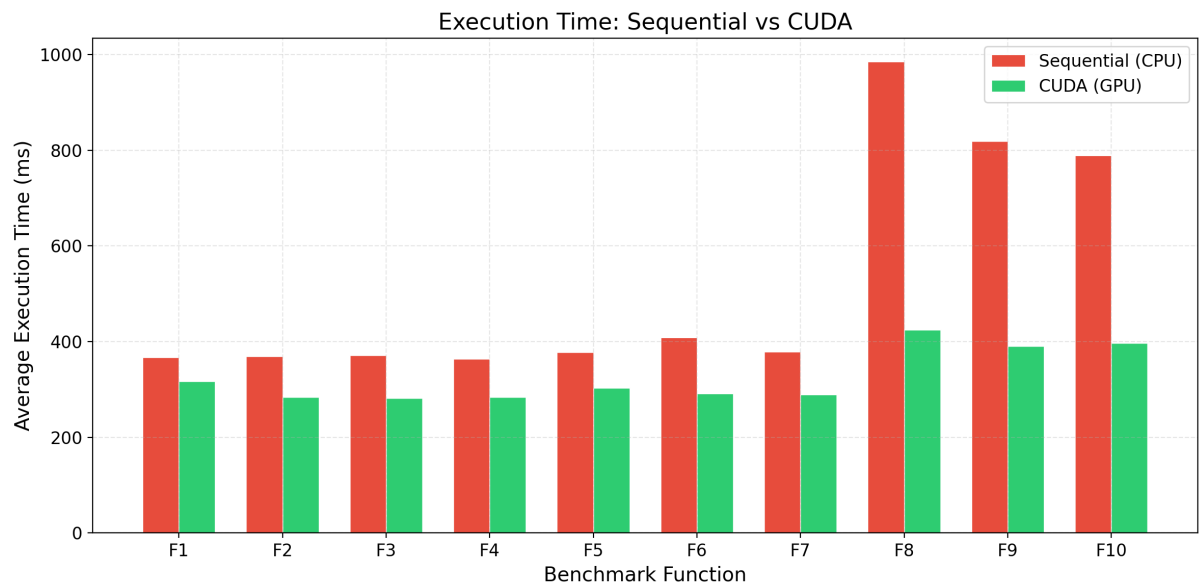


Figure 9: Average execution time comparison between Sequential (CPU) and CUDA (GPU).

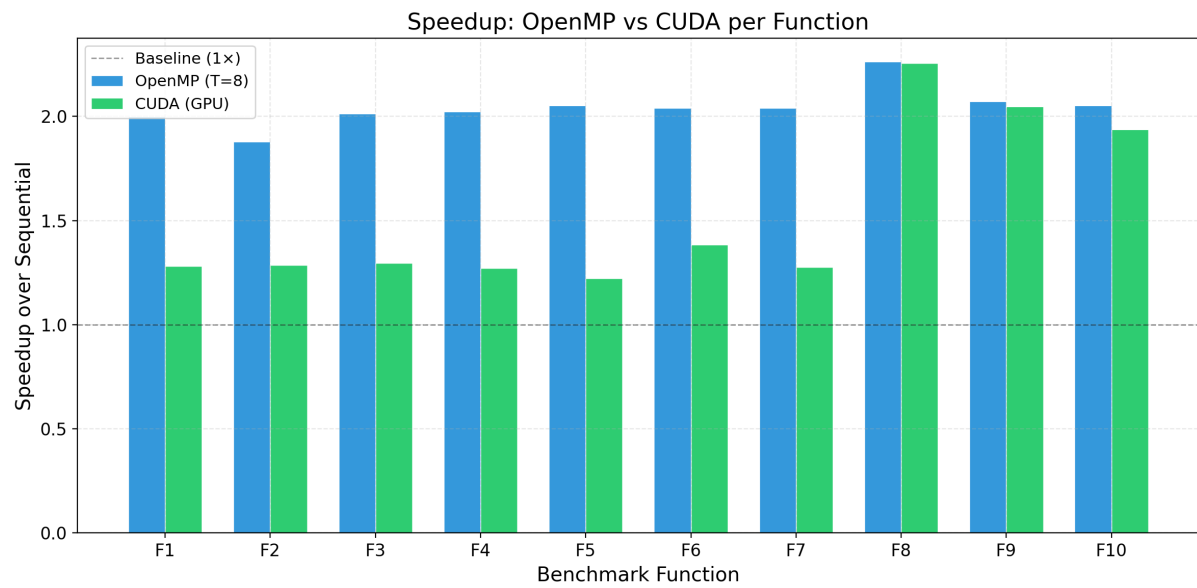


Figure 10: Speedup comparison: OpenMP (at T=8) versus CUDA per function.

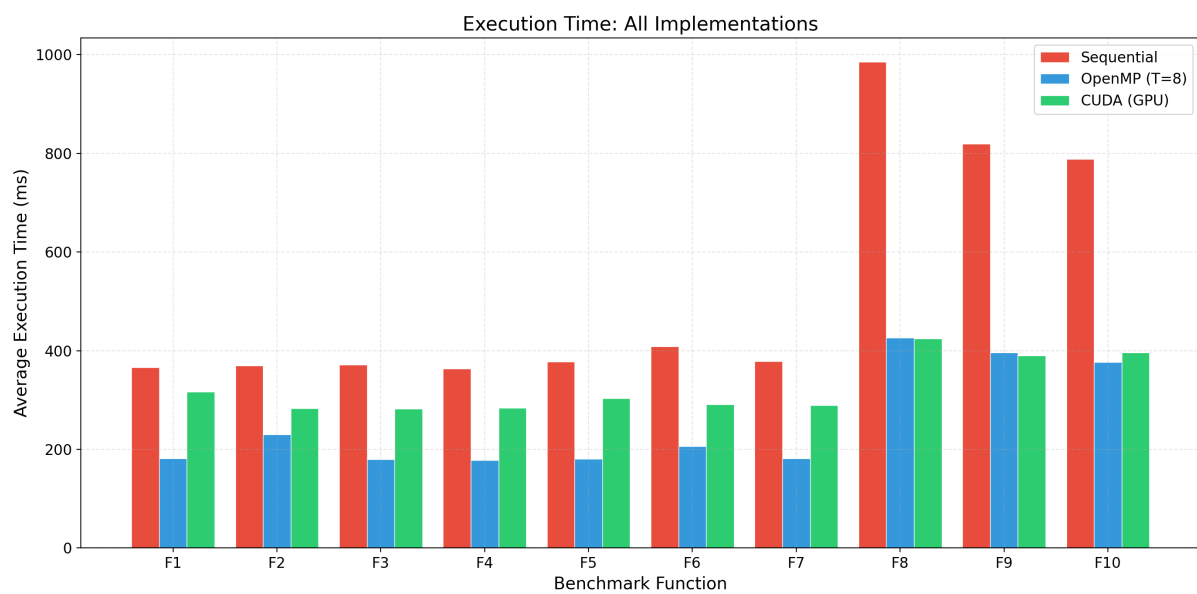


Figure 11: Combined execution time comparison across all three implementations (Sequential, OpenMP T=8, CUDA).

5.5 Scalability Analysis

5.5.1 Marginal Speedup Gain

Table 7: Marginal speedup gain when doubling threads.

Transition	Marginal Gain	Linear Scaling Ratio
$T = 1 \rightarrow T = 2$	+0.491	67.7%
$T = 2 \rightarrow T = 4$	+0.531	43.7%
$T = 4 \rightarrow T = 8$	+0.295	16.9%
$T = 8 \rightarrow T = 16$	-1.678	-82.1%

The negative marginal gain at $T=16$ confirms that thread overhead dominates at high thread counts.

5.5.2 Workload Size Impact

- **Small workloads** ($\leq 200K$ operations): Average speedup = $1.107\times$
- **Large workloads** ($> 500K$ operations): Average speedup = $1.291\times$

Larger workloads benefit more from parallelization, as expected from Gustafson's Law perspective.

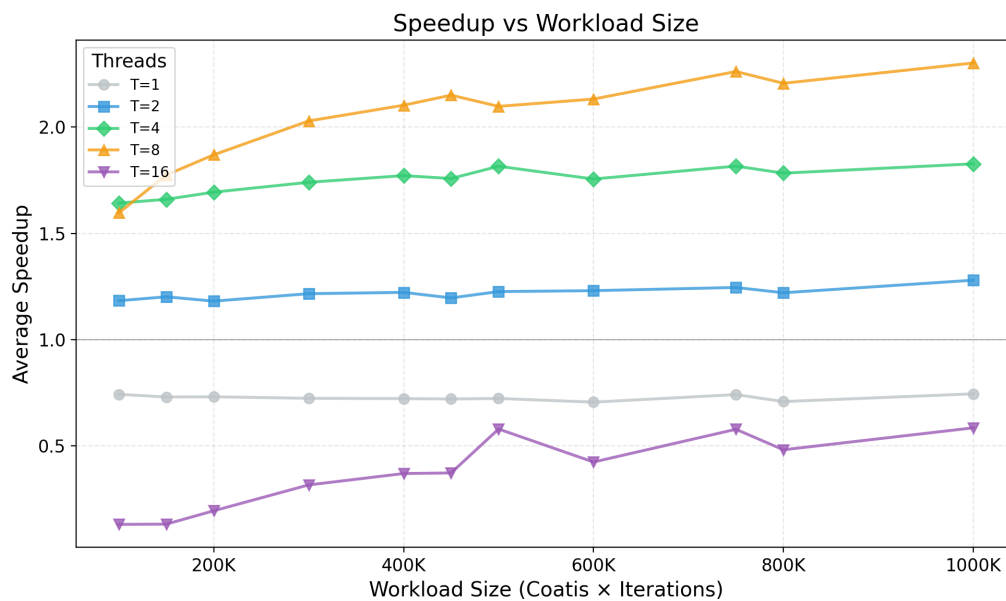


Figure 12: Speedup versus total workload size (Coatis $\times$ Iterations). Larger workloads consistently yield better parallel speedup.

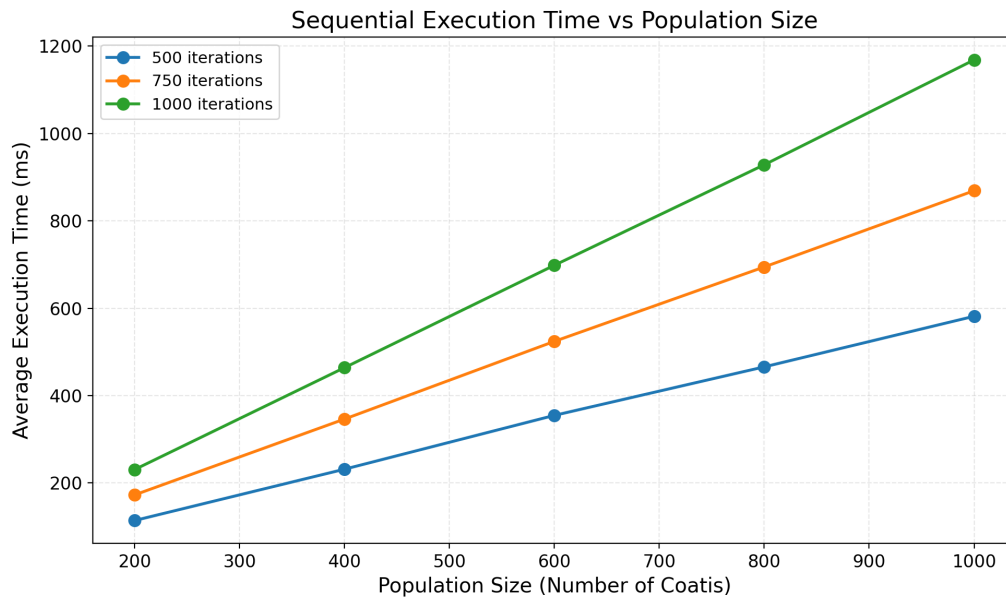


Figure 13: Sequential execution time scaling with population size.

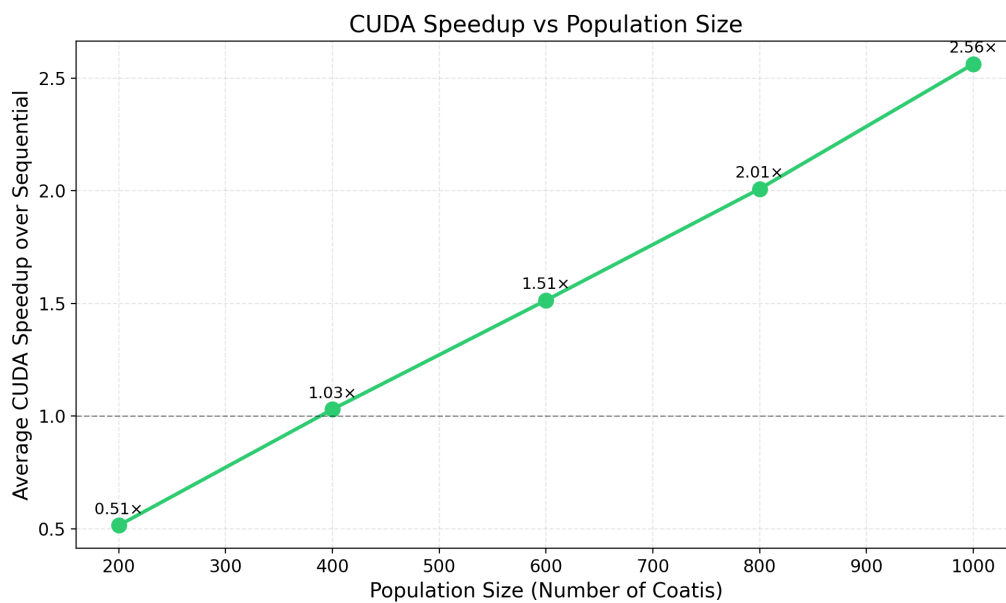


Figure 14: CUDA speedup versus population size. GPU acceleration becomes more effective as the amount of parallel work increases.

5.6 Amdahl's Law Analysis

Using the observed speedup at $T=16$, we estimate the serial fraction f for each function via Amdahl's Law:

$$S(p) = \frac{1}{f + \frac{1-f}{p}} \implies f = \frac{1/S - 1/p}{1 - 1/p}$$

Table 8: Estimated serial fraction from Amdahl’s Law ($T=16$).

Function	Serial Fraction (f)	Max Theoretical Speedup
All functions (F1–F10)	≈ 1.00	$\approx 1.0\times$

The estimated serial fraction is very high (≈ 1.0) when measured at $T=16$. This is partly an artifact of the severe overhead at $T=16$ and partly reflects the fundamental structure of COA: each iteration depends on the global best from the previous iteration, creating a serial dependency chain.

Important nuance: At $T=4$ and $T=8$, meaningful speedup is achieved ($1.75\text{--}2.04\times$), suggesting that the “effective” serial fraction is lower at moderate thread counts. The high serial fraction at $T=16$ is dominated by overhead rather than algorithmic seriality.

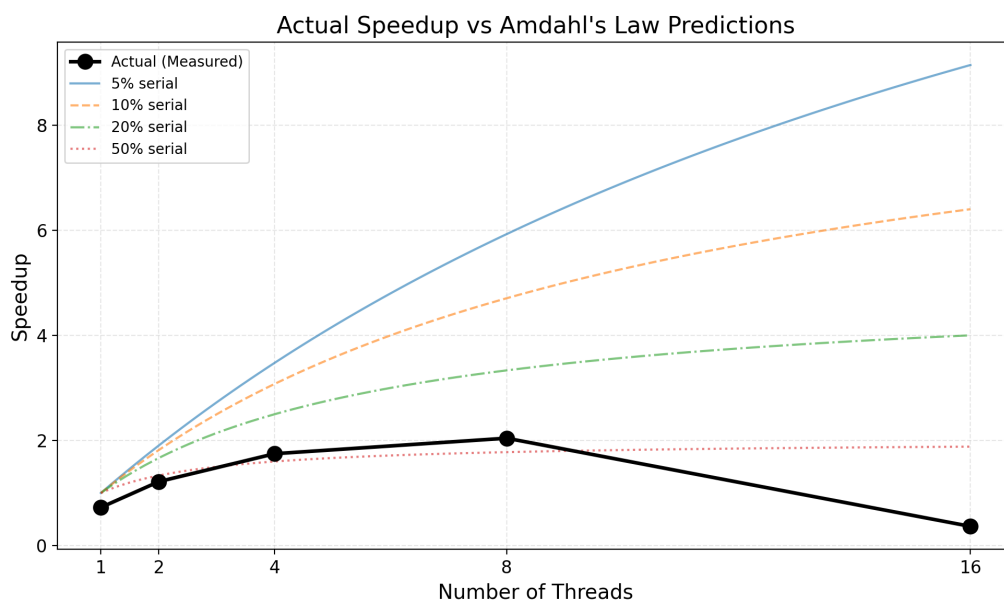


Figure 15: Actual measured average speedup compared against Amdahl’s Law theoretical curves for various serial fractions.

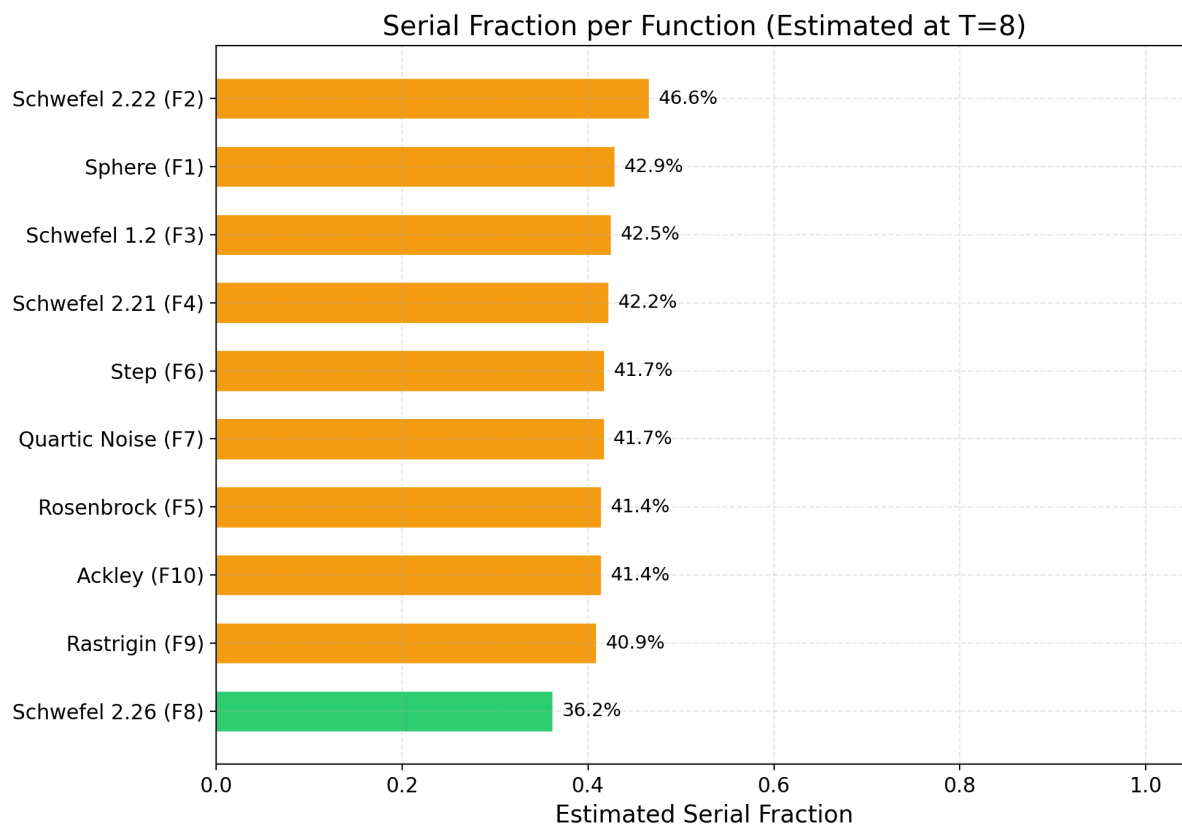


Figure 16: Estimated serial fraction for each benchmark function, calculated at T=8.

5.7 Overhead and Anomaly Analysis

Table 9: Slowdown analysis summary.

Metric	Value
Total slowdown cases	1,549 / 3,750 (41.3%)
T=1 slowdown rate	99.7% (OpenMP overhead)
T=2 slowdown rate	4.4%
T=8 slowdown rate	2.4%
T=16 slowdown rate	100.0% (always slower)
Super-linear cases	2 (0.1% — likely cache effects)

Worst slowdowns occurred at T=16 with small populations (C=200), where thread management overhead vastly exceeds the benefit of parallelism. The worst case (F2, C200, I500, T=8) showed a $0.055\times$ speedup ($18\times$ slower than sequential).

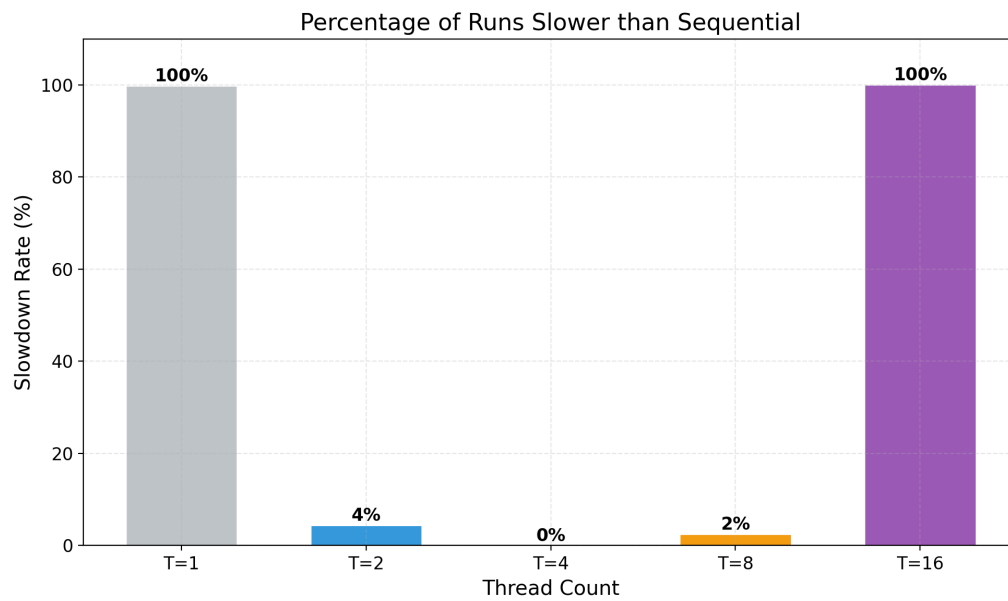


Figure 17: Percentage of runs that were slower than the sequential baseline at each thread count.

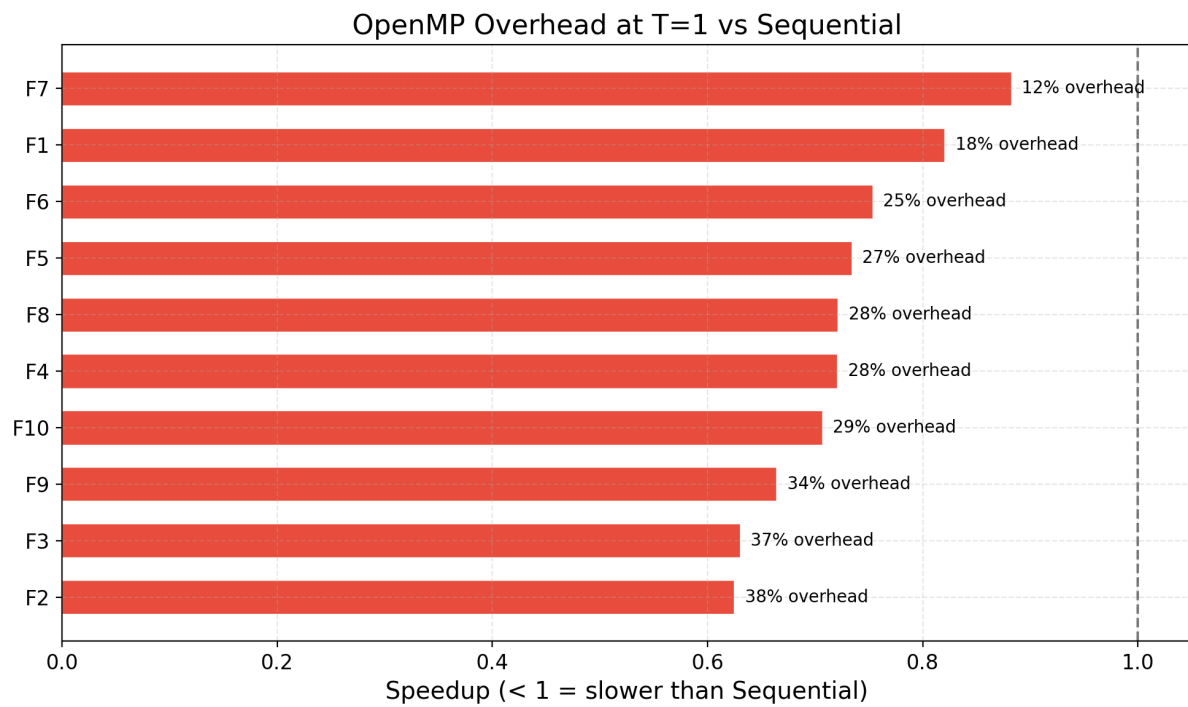


Figure 18: OpenMP runtime overhead measured at T=1 (compared to sequential baseline).

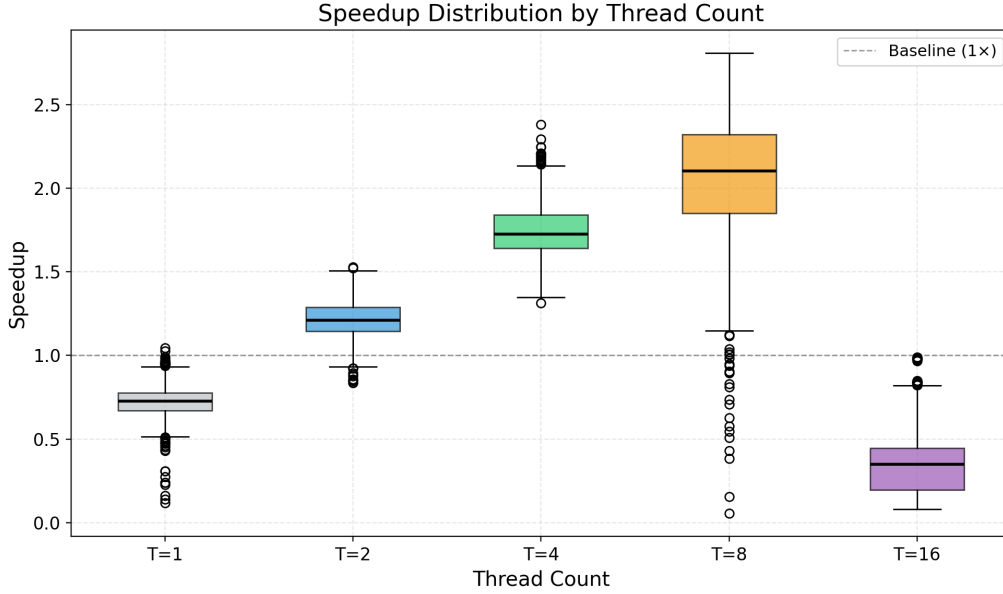


Figure 19: Box plot of speedup distribution by thread count, showing the variance across all configurations.

6 Discussion

6.1 Why is Parallel Efficiency Low?

Several factors contribute to the limited parallel scalability of COA:

1. **Serial iteration dependency:** Each iteration updates the global best, which all threads need at the start of the next iteration. This creates an inherent serial bottleneck.
2. **Lightweight fitness functions:** For $D=30$, each fitness evaluation involves only ~ 30 – 60 floating-point operations. This provides very little computational work to amortize thread synchronization overhead.
3. **OpenMP runtime overhead:** Thread creation, barrier synchronization, and work-sharing constructs add fixed costs per iteration.
4. **Memory access patterns:** The 2D population matrix (`vector<vector<double>>`) has poor spatial locality for multi-threaded access.

6.2 CUDA vs OpenMP

CUDA outperforms OpenMP for compute-heavy functions because:

- The GPU provides *thousands* of hardware threads vs. ≤ 16 CPU threads.
- Flattened memory layout (`X[i * D + j]`) enables coalesced global memory access.
- Per-thread register-based computation avoids shared-memory contention.

- For functions involving transcendental operations, GPU SFUs (Special Function Units) provide hardware-accelerated `sin/cos/exp`.

However, CUDA is limited by:

- Kernel launch overhead ($\sim 5\text{--}20\ \mu\text{s}$ per launch, 4–5 launches per iteration).
- Host-device memory transfers for best-index updates.
- The outer iteration loop remaining on the CPU.

6.3 Recommendations

Based on our findings:

1. **Use 4–8 OpenMP threads** for the best speedup-to-efficiency trade-off.
2. **Use CUDA for compute-heavy functions** (F8–F10) with large populations (≥ 600).
3. **Add an OpenMP if clause** to disable parallelism for small workloads ($N < 400$).
4. **Consider algorithmic restructuring:** batch multiple iterations or use asynchronous updates to reduce serial dependencies.
5. **CUDA occupancy tuning:** experiment with different block sizes and explore shared-memory caching for the Iguana vector.

7 Convergence and Final Efficiency Analysis

This section presents the convergence behavior of the algorithm and a final parallel efficiency heatmap.

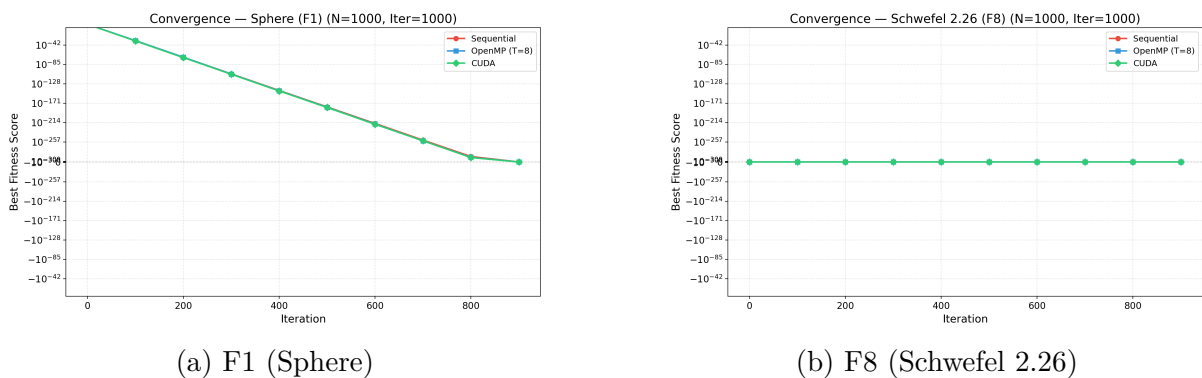


Figure 20: Convergence curves for $N = 1000$ coatis and $T = 1000$ iterations. All implementations converge to similar final solutions, confirming correctness.

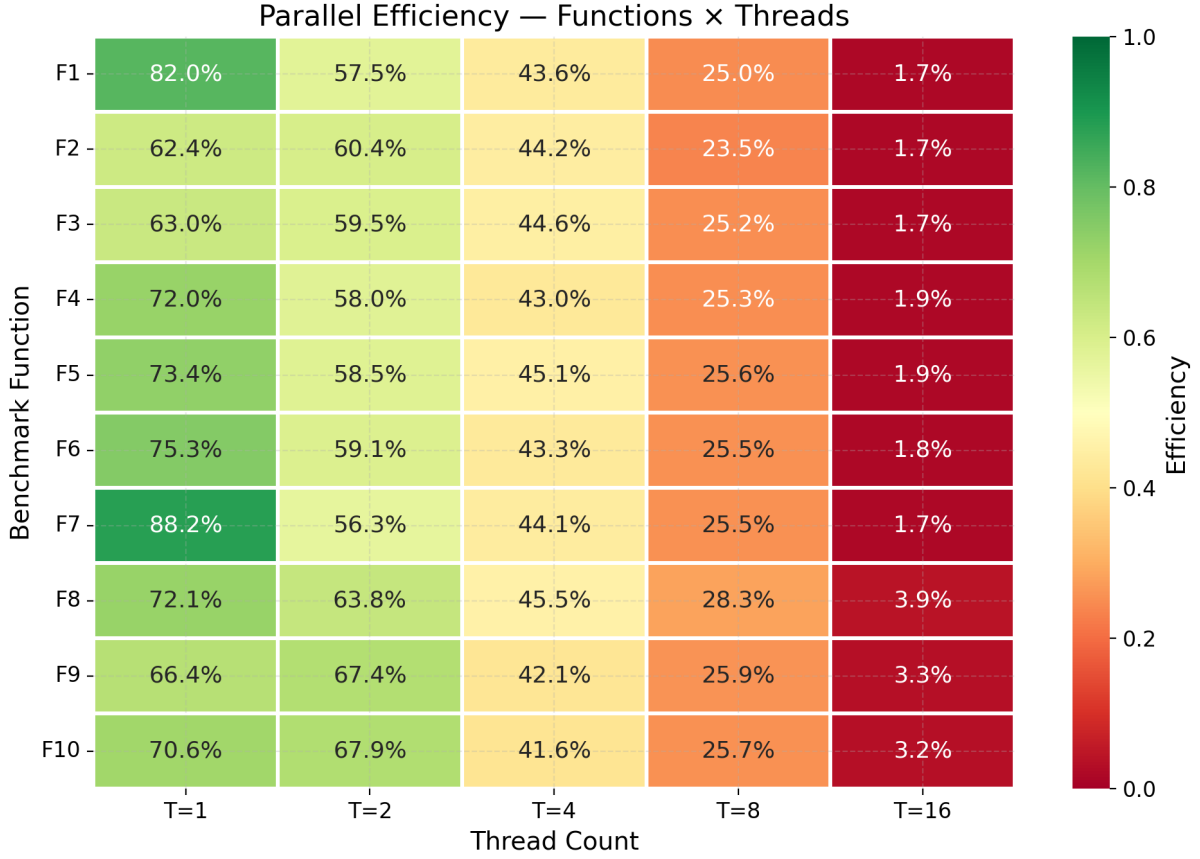


Figure 21: Parallel efficiency heatmap across functions and thread counts.

8 Conclusion

This project provides a thorough HPC analysis of the Coati Optimization Algorithm across three computational paradigms. Our key conclusions are:

1. **COA is parallelizable, but with limitations:** The inherent serial dependency between iterations limits maximum speedup regardless of the number of processors.
2. **Optimal OpenMP configuration:** 4–8 threads provide the best balance of speedup (1.75–2.04 $\times$) and efficiency (25–44%).
3. **GPU acceleration is function-dependent:** CUDA achieves up to 4.36 $\times$ speedup for compute-intensive functions but only 1.2–1.4 $\times$ for lightweight ones.
4. **Thread management overhead is significant:** OpenMP imposes $\sim 27\%$ overhead even at T=1, and performance degrades below sequential at T=16.
5. **Larger workloads scale better:** Consistent with both Amdahl’s and Gustafson’s Laws.

8.1 Future Work

- **CUDA occupancy tuning:** Optimize block sizes and shared memory usage.

- **Multi-GPU experiments:** Distribute population across multiple GPUs.
- **Asynchronous COA:** Relax the synchronous iteration structure to improve parallel fraction.
- **Memory coalescing analysis:** Profile and optimize GPU memory access patterns.
- **Higher dimensions:** Test with $D = 100, 500, 1000$ to increase per-thread work.
- **Comparison with other metaheuristics:** Benchmark parallel COA against parallel PSO, GA, and DE.

References

- [1] Dehghani, M., Montazeri, Z., Trojovská, E., & Trojovský, P. (2023). *Coati Optimization Algorithm: A new bio-inspired metaheuristic algorithm for solving optimization problems*. Knowledge-Based Systems, 259, 110011.
- [2] Amdahl, G. M. (1967). *Validity of the single processor approach to achieving large scale computing capabilities*. Proceedings of the April 18-20, 1967, Spring Joint Computer Conference, 483–485.
- [3] Gustafson, J. L. (1988). *Reevaluating Amdahl’s law*. Communications of the ACM, 31(5), 532–533.
- [4] OpenMP Architecture Review Board. (2021). *OpenMP Application Programming Interface, Version 5.2*. <https://www.openmp.org/specifications/>
- [5] NVIDIA Corporation. (2023). *CUDA C++ Programming Guide*. <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>